

VigilantRAG

Technical Reference Manual & Interview Preparation Guide

Version 1.0.0

1. Executive Summary & Project Vision

VigilantRAG is an advanced, production-grade **Self-Correcting Retrieval-Augmented Generation (RAG)** engine. Traditional RAG setups suffer from critical vulnerabilities when deployed in production:

- **Retrieval Noise:** Semantic search models retrieve irrelevant context because queries are often short or contain keywords not matching the document corpus.
- **Hallucinations:** Large Language Models (LLMs) compose answers that sound authoritative but contain statements completely unsupported by the retrieved documents.
- **Static Execution:** Traditional RAG loops are linear (Retrieve → Generate). If retrieval returns junk context, the LLM generates junk answers, with no mechanism to self-correct.

VigilantRAG addresses these challenges by introducing an active, self-correcting feedback loop:

1. It evaluates the quality of retrieved contexts using a **Cross-Encoder re-ranker**. If context relevance drops below a threshold, it triggers **Query Expansion** (synonyms/LLM rephrase) to fetch better data.
2. It runs a **Natural Language Inference (NLI)** auditor against the LLM's draft answer.
3. If a contradiction is detected, the draft is blocked, LLM parameters are adjusted (increasing temperature, adding strict prompt constraints), and the answer is regenerated until it passes the audit.

2. Tech Stack Architecture & Tradeoffs

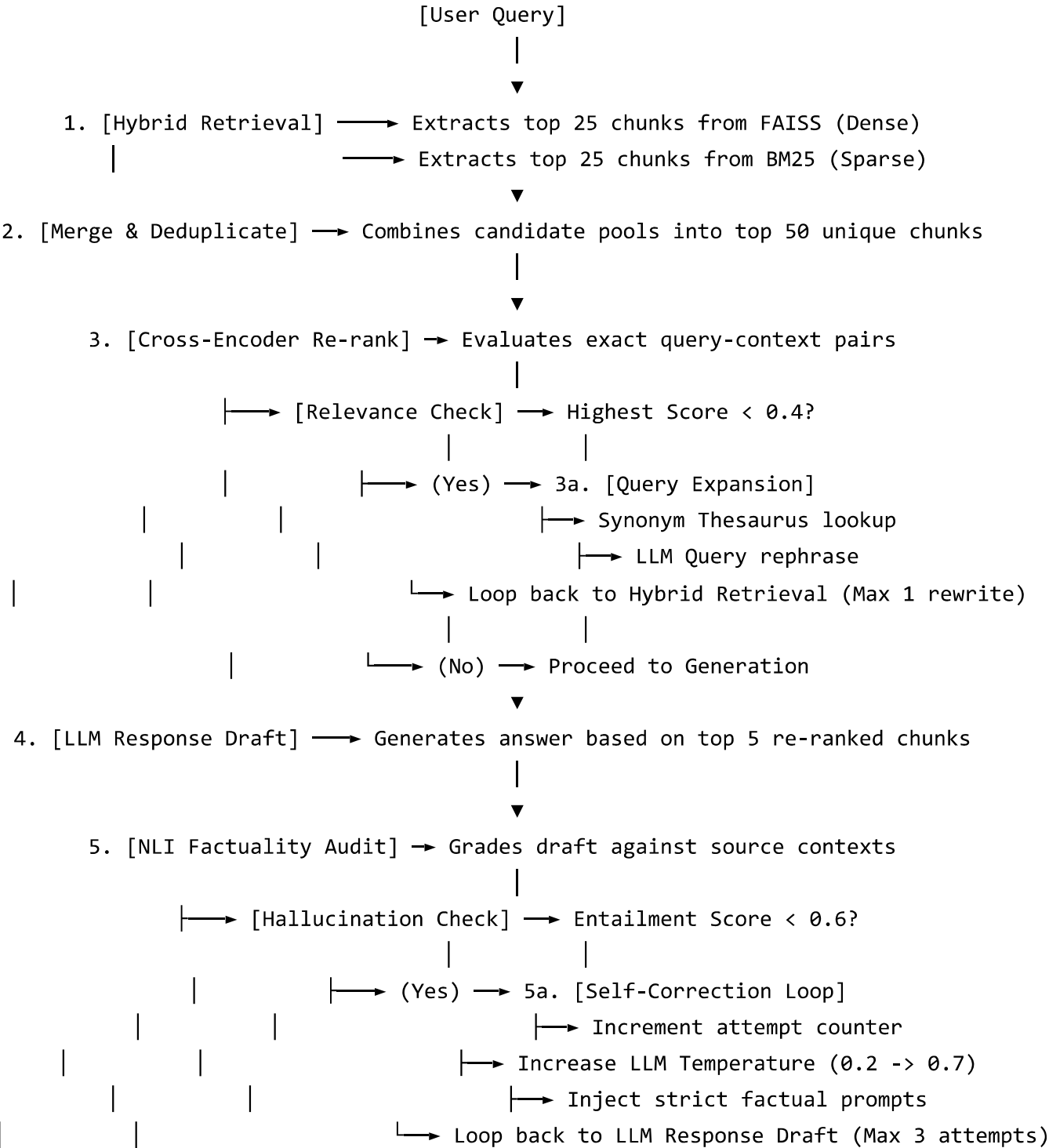
This project uses a fully local, resource-efficient stack designed to run on low-tier CPUs (such as free Hugging Face instances) while maintaining high precision. Below is the technical breakdown and design tradeoffs:

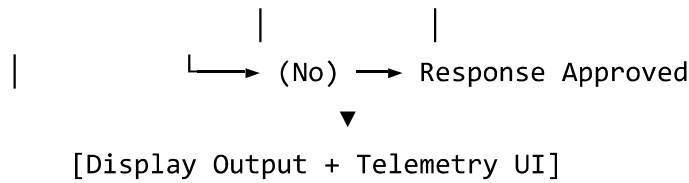
Component	Selected Tech	Why We Used It	Why Not Alternatives?
Programming Language	Python 3.11	Standard for AI/ML engineering. Native libraries for tensor operations, vector spaces, and transformers.	<i>Node.js / Go</i> : Immature tooling for local model weights and execution.
Web Server	FastAPI	Asynchronous, highly performant, handles concurrent loops, native Pydantic validation, auto-generated OpenAPI docs.	<i>Flask</i> : Synchronous blockages, requires manual plugins. <i>Django</i> : Heavyweight, bloated for a single-page engine.
Dense Index	FAISS	High-performance local vector similarity search. In-memory, runs on CPU without background services.	<i>Pinecone/Milvus</i> : Costly, require internet APIs or complex docker containers.
Sparse Index	Rank-BM25	Term-frequency matching. Essential for exact names, specific serial numbers, and technical jargon.	<i>SQL LIKE</i> : Inefficient, does not compute statistical term frequency.
Bi-Encoder	all-MiniLM-L6-v2	384-dimensional dense embedding model. Extremely lightweight (90MB), runs in milliseconds on CPU.	<i>OpenAI embeddings</i> : Introduces network dependency, API costs, and privacy concerns.
Re-ranker Model	ms-marco-MiniLM-L6-v2	Cross-Encoder model. Scores query-document pairs jointly to capture deep semantic relevance.	<i>Cosine Similarity alone</i> : Fails to capture subtle word interactions.
Factual Auditor	nli-deberta-v3-xsmall	Natural Language Inference classifier. Detects logical conflicts between answer and source context.	<i>LLM-as-a-Judge</i> : Slow, non-deterministic, expensive, and can hallucinate the audit.

Component	Selected Tech	Why We Used It	Why Not Alternatives?
Generative LLM	Qwen2.5-0.5B-Instruct	Lightweight, 0.5B instruction-tuned model. Runs locally on CPU, handles system prompts cleanly.	<i>Llama-3-8B</i> : Too heavy for basic servers (requires dedicated GPU).

3. Pipeline Architecture & Data Flow

The following text diagram details how data flows through the multi-stage engine for every query:





4. Detailed Codebase File Directory

Here is what each file in the workspace is responsible for:

- `src/config.py` : Configures model names, directory paths (such as the `data/` folder), default thresholds (relevance=0.40, NLI=0.60), and system prompt templates.
- `src/retriever.py` : Manages document ingestion. Chunks text with configurable overlap, runs the FAISS vector index, handles BM25 tokenization, and merges results.
- `src/reranker.py` : Computes joint query-context logits using the Cross-Encoder model and sorts chunks by score.
- `src/query_expansion.py` : Performs query rewriting. Contains synonym rules (e.g. mapping "wfh" → "remote", "study" → "learning") and fallback LLM rephrasing prompts.
- `src/hallucination_guard.py` : Evaluates the factual alignment of drafts against the context. Outputs entailment, neutral, and contradiction scores.
- `src/llm_client.py` : Manages the local Qwen2.5 model instance, formats instruction prompts, and adjusts temperatures.
- `src/engine.py` : Coordinates the pipeline. Collects telemetry logs (timing metrics, intermediate drafts, scores) and exposes the clean `query()` function.
- `app.py` : FastAPI server file. Sets up CORS, mounts static UI files, and defines API endpoints (`/api/query`, `/api/ingest`, `/api/documents`).

5. Interactive Dashboard UI Walkthrough

The dashboard is built with a premium glassmorphic dark-mode CSS theme. Here is how each visual component works:

A. Sidebar Control Panel

- **Relevance Threshold Slider:** Sets the limit for query correction. If the top document score is below this value, the engine assumes the context is irrelevant and triggers Query Expansion.
- **Entailment Threshold Slider:** The safety threshold. If the NLI entailment score of the generated text falls below this value, the response is discarded as a hallucination.
- **LLM Temperature Slider:** Adjusts randomness. Low temperatures (0.1 - 0.2) force the model to stick strictly to facts. High temperatures (0.6 - 0.8) are automatically triggered during self-correction to help the model escape phrasing loops.
- **Ingest Document Form:** Processes custom documents in real-time, breaking them into overlapping chunks and re-indexing them instantly.

B. Pipeline Trace Map (Timeline)

A visual timeline representing the stages of execution. Clicking on any step dynamically populates the details panel on the right with internal execution variables:

- **Input Query:** Shows the raw user question.
- **Hybrid Search:** Displays side-by-side list of FAISS semantic matches and BM25 exact keyword matches.
- **Re-ranking:** Lists sorted chunks with their Cross-Encoder relevance scores.
- **Query Expansion:** Shows synonym lookup logs and the rewritten query string.
- **LLM Generate:** Shows the raw template prompts, formatting, and intermediate drafts.
- **NLI Audit:** Displays NLI scores (Entailment, Neutral, Contradiction) as progress bars.

C. Telemetry & Answer Cards

- **Answer Card (Left):** Renders the final answer. Includes color-coded status badges: Green (*Verified Factual*), Orange (*Corrected & Verified*), or Red (*Blocked*). Displays execution duration.
- **Stage Details (Right):** An interactive display showcasing the mathematical output, synonyms, or NLI charts corresponding to the clicked timeline stage.

6. Key Technical Definitions

- **Vector Embedding:** A dense numerical list representing a text's meaning in a multi-dimensional space, where similar concepts sit closer together.
- **Bi-Encoder:** Encodes queries and documents separately. Super-fast at retrieval over large collections, but less precise at complex semantic matching.
- **Cross-Encoder:** Encodes queries and documents together. Highly accurate at scoring semantic matching because it applies self-attention over both query and document text simultaneously. Too slow to run over millions of files, so used as a re-ranker.
- **Natural Language Inference (NLI):** The task of determining the logical relationship between a Premise (source context) and a Hypothesis (AI answer). The output categories are:
 - *Entailment:* The hypothesis is logically supported by the premise.
 - *Contradiction:* The hypothesis contradicts or asserts facts not in the premise.
 - *Neutral:* The hypothesis is unrelated to the premise.

7. Interview Q&A Cheat Sheet

Q1: What happens when the RAG retriever fails to find relevant documents on the first pass?

A: VigilantRAG monitors the Cross-Encoder score of the top-ranked chunk. If it falls below the relevance threshold (e.g. 0.40), the engine flags it as irrelevant context. It stops, expands the query with domain synonyms or an LLM rewrite, and re-executes the search. This prevents the model from generating answers based on unrelated, garbage context.

Q2: Why use an NLI model for hallucination checking instead of another LLM query?

A: NLI models are specialized classifiers trained specifically to grade logical entailment. They are deterministic, fast (running in milliseconds on CPU), and output exact probability logits. Using an LLM to check another LLM is slow, expensive, prone to prompt injection, and suffers from the same hallucination issues it is trying to detect.

Q3: How does the system handle "unanswerable" questions where facts do not exist in the corpus?

A: When a user asks an unanswerable question, the initial relevance check fails, triggering query expansion. If the second retrieval pass still fails to find relevant chunks (relevance remains < 0.40), the engine instructs the LLM to output a fallback response ("I do not have sufficient information in the context to answer this question.") which passes the NLI check because it makes no active assertions.

Q4: Why run the models locally inside the container instead of calling API endpoints?

A: Local model execution ensures absolute data privacy, eliminates API costs, avoids network latency/outages, and ensures the application is completely self-contained. By using optimized, small-footprint models (like MiniLM and Qwen-0.5B), we run inference quickly on basic CPU servers.